₹

The Code, Explained

A line-by-line walkthrough of the Income Tax Evaluation System — written so that a reader with no programming background can follow every part.

Document	Technical explainer & code review
Audience	Everyone — no coding knowledge assumed
Covers	Visual Basic (VB.NET) · Oracle SQL · Web edition
Version	1.0

1 · How to read this document

This document opens up the entire application and explains it in plain English. You do not need to know Visual Basic, SQL, or any programming language.

Every code section follows the same three-step rhythm, so you always know what you are looking at:

1. **What it does** — one plain sentence, before any code appears.
2. **The code** — shown on a dark background with line numbers down the side.
3. **Line by line** — a table that takes each line number and explains it in ordinary words.

> **PLAIN-ENGLISH BOX**
> Whenever a technical idea appears for the first time, a box like this explains it with an everyday comparison. You can safely skip the dark code blocks and still understand the whole system by reading the tables and boxes.

ANALOGY
Think of the application as a
TAX OFFICE
. The *database* is the filing cabinet, the *code* is the clerk who knows the rules, and the *screen* is the counter where the public interacts. Most of this document explains what the clerk does.

Contents

2 · What the system actually does

A tax practitioner looks after many clients. For each client, every year, they must record income, work out the tax, file a return, and sometimes file a corrected ("revised") return. If the client is a firm, they must also keep three account books. This system does all of that on a computer instead of on paper.

MODULE	IN PLAIN WORDS
Client Information	The address book. One record per client, identified by their PAN.
Original Return	The tax return filed for a year, with the tax worked out automatically.
Revised Return	A correction to a return already filed.
Trading Account	For firms: goods bought and sold, ending in gross profit.
Profit & Loss Account	For firms: running expenses against income, ending in net profit.
Balance Sheet	For firms: what the firm owns vs. what it owes.
Reports	Printable summaries for the client file.

3 · The big picture — how the parts fit together

ANALOGY
When you use the system you are the
CUSTOMER AT THE COUNTER
. Your browser sends a request; the
SERVER
(the clerk) reads it, looks things up in the
DATABASE
(the filing cabinet), works out the answer, and sends back a page.

The project ships the same system twice, because it must satisfy two different needs:

EDITION	BUILT WITH	WHY IT EXISTS
Production	Visual Basic (VB.NET) + Oracle database	The real, deployable system as specified.
Web edition	HTML + JavaScript, data kept in the browser	So anyone can open a link and try it — no install, no database.

Both editions use the **same seven modules**, the **same five tables**, and the **same tax rules**. The explanations below therefore apply to both.

The folders

FOLDER	WHAT LIVES THERE
`src/`	The Visual Basic production application.
`webapp/`	The web edition (what the live demo runs).
`database/`	Four SQL scripts that build the Oracle database.
`docs/`	This document and the user guide.
`dist/`	The web edition packed into one self-contained file, ready to publish.
`demo-video/`	The narrated demo video, its slides and the scripts that build it.

> **WHY TWO FILES PER SCREEN?**
> In Visual Basic web projects each screen is two files: a `.aspx` file that describes *what the page looks like*, and a `.aspx.vb` file (the "code-behind") that describes *what the page does*. Design and behaviour are kept apart on purpose.

4 · The filing cabinet — the Oracle database

Everything the system remembers lives in five tables. A **table** is just a grid: columns are the fields, rows are the records.

TABLE	ONE ROW MEANS…
CLIENT_RECORD	one client
INCOME_TAX_RECORD	one return, for one client, for one year
TRADING_ACCOUNT	one firm's trading account for one year
PL_ACCOUNT	one firm's profit & loss account for one year
BALANCE_SHEET	one firm's balance sheet for one year

4.1 The client table, line by line

What it does: creates the table that stores clients, and refuses to store one that is invalid.

DATABASE/02_CREATE_SCHEMA.SQL

```
1 CREATE TABLE CLIENT_RECORD (
2   PAN                  VARCHAR2(10) CONSTRAINT PK_CLIENT_RECORD PRIMARY KEY,
3   CLIENT_NAME          VARCHAR2(30) NOT NULL,
4   DOB                  DATE,
5   INDV_HUF_FIRM_AOP_LA  NUMBER(1) DEFAULT 0
6     CONSTRAINT CK_CATEGORY CHECK (INDV_HUF_FIRM_AOP_LA BETWEEN 0 AND 4),
7   CONSTRAINT CK_PAN_FORMAT CHECK (REGEXP_LIKE(PAN, '^[A-Z]{5}[0-9]{4}[A-Z]$'))
8 );
```

LINE	WHAT IT MEANS IN PLAIN WORDS
1	"Make a new grid called CLIENT_RECORD."
2	PAN holds up to 10 characters and is the **primary key** — the unique label for the row. Two clients can never share a PAN.
3	The name may be up to 30 characters and **cannot be left blank**.
4	Date of birth is stored as a real date, not text — so the computer can compare and sort dates.
5–6	Category is a single digit. The **CHECK** rule allows only 0–4 (Individual, HUF, Firm, AOP, Local Authority). Anything else is rejected.
7	A pattern rule: PAN must be 5 letters, 4 digits, 1 letter — the real PAN format. A typo is refused *by the database itself*.

> **WHY THIS MATTERS**
> These rules live in the database, not just on the screen. Even if someone bypassed the app entirely, bad data still could not get in. This is called *defence in depth*.

4.2 Linking tables together

DATABASE/02_CREATE_SCHEMA.SQL — INCOME_TAX_RECORD

```
1 CONSTRAINT PK_INCOME_TAX_RECORD PRIMARY KEY
2   (PAN, ASSES_YEAR_1, ASSES_YEAR_2, RETURN_ORIGINAL_REVISED),
3 CONSTRAINT FK_INCOME_TAX_RECORD FOREIGN KEY (PAN)
4   REFERENCES CLIENT_RECORD (PAN) ON DELETE CASCADE
```

LINE	WHAT IT MEANS IN PLAIN WORDS
1–2	A return is identified by four things together: which client, which year (from and to), and whether it is the original or the revised one. This is exactly why one client can have both an original *and* a revised return for the same year — but never two of the same kind.
3	A **foreign key**: the PAN here must already exist in CLIENT_RECORD. You cannot file a return for a client who does not exist.
4	**ON DELETE CASCADE**: if a client is deleted, all their returns are deleted automatically — no orphaned paperwork left behind.

5 · The tax engine, line by line

This is the heart of the system: given the income, work out the tax. It is written twice — once in Visual Basic for the production app, once in JavaScript for the web edition — and both follow identical rules so the numbers always agree.

5.1 The slab table

ANALOGY

Indian income tax works like a **STAIRCASE**, not a single rate. The first ₹2.5 lakh is free; the next slice is taxed at 5%; the next at 20%; anything above at 30%. You never pay one flat rate on everything — only each slice at its own rate.

SRC/ITEMS/APP_CODE/TAXENGINE.VB

```
1 Private Shared ReadOnly SlabUpto() As Decimal = {250000D, 500000D, 1000000D, Decimal.MaxValue}
2 Private Shared ReadOnly SlabRate() As Decimal = {0D, 0.05D, 0.2D, 0.3D}
3
4 Public Shared Function SlabTax(income As Decimal) As Decimal
5     Dim tax As Decimal = 0D, lower As Decimal = 0D
6     For i = 0 To SlabUpto.Length - 1
7         If income > lower Then
8             Dim band = Math.Min(income, SlabUpto(i)) - lower
9             tax += band * SlabRate(i)
10            lower = SlabUpto(i)
11        Else
12            Exit For
13        End If
14    Next
15    Return Math.Round(tax)
16End Function
```

LINE	WHAT IT MEANS IN PLAIN WORDS
1	The ceilings of each step: ₹2,50,000 / ₹5,00,000 / ₹10,00,000 / no limit. (`D` just tells VB "this is an exact decimal number" — money must never be stored as an approximate value.)
2	The rate for each matching step: 0%, 5%, 20%, 30%.
4	Start of the calculator. It takes one number in (the income) and gives one number back (the tax).
5	`tax` is the running total, starting at zero. `lower` remembers the bottom of the current step, also starting at zero.
6	"Walk up the staircase, one step at a time."
7	Only bother if the income actually reaches this step.
8	Work out how much income falls *inside this step*: whichever is smaller — the income or the step's ceiling — minus the step's floor.
9	Tax that slice at this step's rate and add it to the running total.
10	Move the floor up, ready for the next step.
11–13	If the income never reached this step, stop — every higher step is empty too.
15	Round to whole rupees and hand the answer back.

A worked example — salary of ₹12,00,000

STEP	SLICE OF INCOME	RATE	TAX ON THE SLICE
0 – 2,50,000	2,50,000	0%	₹0
2,50,000 – 5,00,000	2,50,000	5%	₹12,500
5,00,000 – 10,00,000	5,00,000	20%	₹1,00,000
10,00,000 – 12,00,000	2,00,000	30%	₹60,000

Tax on total income	**₹1,72,500**
Add: Health & Education Cess @ 4%	₹6,900
Total tax payable	**₹1,79,400**

VERIFIED

The live application was tested with exactly this input and displayed **₹1,79,400** — matching the hand calculation above.

5.2 The full computation

What it does: takes every income figure, applies the rules in the legal order, and returns the final position — pay this much, or get this much back.

SRC/ITEMS/APP_CODE/TAXENGINE.VB — COMPUTE()

```
1 Dim gross = r.IncomeFromSalary + r.IncomeFromHouseProperty + r.IncomeFromBusiness +
2             capital + r.IncomeFromOtherSources + r.OtherPersonIncome
3 Dim deductions = Math.Min(r.DeductionUnderVIA, gross)
4 Dim totalIncome = Math.Max(0D, gross - deductions)
5
6 Dim taxNormal = If(isFirm, Math.Round(totalIncome * FIRM_RATE), SlabTax(normalBase))
7 Dim rebate As Decimal = 0D
8 If Not isFirm AndAlso totalIncome <= REBATE_LIMIT Then rebate = Math.Min(taxOnTotal, REBATE_MAX)
9 Dim taxPayable = Math.Max(0D, taxOnTotal - rebate)
10Dim surcharge = Math.Round(taxPayable * SurchargeRate(totalIncome))
11Dim cess = Math.Round((taxPayable + surcharge) * CESS_RATE)
12Dim totalTax = taxPayable + surcharge + cess
13Dim balance = netTax + r.InterestPayable - r.TdsAtSource - r.AdvanceTaxPaid - r.SelfAssessmentPaid
```

LINE	WHAT IT MEANS IN PLAIN WORDS
1–2	Add up every kind of income: salary, rent from house property, business profit, capital gains, other sources, and any income of another person that the law says must be counted here. This total is the **Gross Total Income**.
3	Subtract savings-based deductions (Chapter VI-A, e.g. insurance, PF). `Math.Min` is a safety guard: you can never deduct more than you earned.
4	The result is the **Total Income** — the figure tax is actually charged on. `Math.Max(0, …)` stops it ever going negative.
6	A fork in the road: a **firm** pays a flat 30%; an **individual** goes up the staircase from §5.1.
7–8	The small-income rebate (section 87A): if total income is within the limit, knock off up to ₹12,500 — but never more tax than is owed.
9	Tax after the rebate; again floored at zero.
10	**Surcharge** — an extra percentage on the tax itself for very high incomes.
11	**Cess** — 4% charged on (tax + surcharge), funding health and education.
12	Everything added together: the total tax payable.
13	Finally, subtract what has already been paid (tax deducted at source, advance tax, self-assessment tax). A **positive** answer means the client still owes money; a **negative** answer means a refund is due.

> **IMPORTANT**
> The rates here follow the old-regime slabs for AY 2024-25 and are intended for demonstration and training. Always check the current Finance Act before using any figure for a real filing.

6 · Security — how passwords are protected

ANALOGY

The system never stores your password. It stores a **FINGERPRINT** of it. A fingerprint can be produced from a person, but you cannot rebuild the person from the fingerprint. When you log in, the system takes a fresh fingerprint and compares the two.

SRC/ITEMS/APP_CODE/SECURITY.VB

```
Public Shared Function Sha256(text As String) As String
    Using sha = SHA256.Create()
        Dim bytes = sha.ComputeHash(Encoding.UTF8.GetBytes(If(text, "")))
        ...
End Function

Public Shared Function Authenticate(username As String, password As String) As String
    Dim row = OracleDb.QueryRow(
        "SELECT FULL_NAME, PASSWORD_HASH, ROLE FROM APP_USER WHERE USERNAME = :u AND IS_ACTIVE = 'Y'",
        OracleDb.PStr("u", username.Trim().ToLowerInvariant()))
    If row Is Nothing Then Return Nothing
    If Not String.Equals(Convert.ToString(row("PASSWORD_HASH")), Sha256(password),
                         StringComparison.OrdinalIgnoreCase) Then
        Return Nothing
    End If
    Return Convert.ToString(row("FULL_NAME"))
End Function
```

LINE	WHAT IT MEANS IN PLAIN WORDS
1–3	The fingerprint machine. Feed it any text, get back a fixed 64-character code (a **SHA-256 hash**). The same text always gives the same code; a one-letter change gives a completely different code.
7	The login check. It is handed a username and password and answers with the person's name if they are genuine — or nothing at all.
8–10	Fetch that user's stored fingerprint from the database. `IS_ACTIVE = 'Y'` means disabled accounts can never log in.
11	No such user? Stop immediately.
12–15	Fingerprint the password that was just typed and compare it with the stored one. If they differ, refuse.
16	Both checks passed — return the person's real name, which the screen then greets them with.

NOTICE WHAT IS MISSING

At no point does the code fetch the real password, because the real password is not stored anywhere. Even someone who stole the whole database would only find fingerprints.

6.1 The audit trail

Every insert, update and delete is recorded automatically by the database itself using a **trigger** — a small piece of code Oracle runs by itself whenever a table changes.

DATABASE/04_TRIGGERS_AUDIT.SQL

```
CREATE OR REPLACE TRIGGER TRG_AUDIT_CLIENT
  AFTER INSERT OR UPDATE OR DELETE ON CLIENT_RECORD
  FOR EACH ROW
BEGIN
  IF INSERTING THEN v_op := 'INSERT'; v_key := :NEW.PAN;
  ...
  INSERT INTO AUDIT_LOG (TABLE_NAME, OPERATION, KEY_VALUE, DB_USER)
  VALUES ('CLIENT_RECORD', v_op, v_key, USER);
END;
```

LINE	WHAT IT MEANS IN PLAIN WORDS

1–3	"Oracle: after anything changes in the client table, for every row affected, run the following."
5	Work out which kind of change happened and which client it was.
7–8	Write a line into the log book: which table, what happened, which record, which database user — with the time stamped automatically.

WHY A TRIGGER, NOT APP CODE?

Because it cannot be bypassed. Even a database administrator changing data directly is recorded.

7 · Talking to the database safely

ANALOGY

Imagine dictating a form to a clerk. If you say "write *Ravi* in the name box", that is safe. If the clerk instead pastes whatever you say straight into the *instructions*, a mischievous person could dictate "…and also shred the cabinet". That attack is called

SQL INJECTION

, and parameters are the fix.

SRC/ITEMS/APP_CODE/ORACLEDB.VB

```
1 Public Shared Function Query(sql As String, ParamArray ps As OracleParameter()) As DataTable
2     Using cn = OpenConnection()
3         Using cmd As New OracleCommand(sql, cn)
4             cmd.BindByName = True
5             If ps IsNot Nothing Then cmd.Parameters.AddRange(ps)
6             Using da As New OracleDataAdapter(cmd)
7                 Dim dt As New DataTable() : da.Fill(dt) : Return dt
8             End Using
9         End Using
10    End Using
11End Function
```

LINE	WHAT IT MEANS IN PLAIN WORDS
1	One reusable helper for every "go and look something up" in the whole application. Writing it once means the safety rules are applied everywhere.
2	Open a connection to Oracle. `Using` is a promise: *whatever happens* — even a crash — the connection will be closed. This prevents the app slowly leaking connections until the database refuses new ones.
3–4	Prepare the instruction. `BindByName` means values are matched by name (`:pan`), not by position — so re-ordering can never silently swap two values.
5	Attach the values **separately** from the instruction. This is the line that defeats SQL injection: user text is only ever data, never commands.
6–7	Run it and pour the results into a grid in memory, then hand that back.

EVERY SINGLE DATABASE CALL IN THIS PROJECT

goes through this helper (or its `Execute` / `Scalar` siblings). There is no place where user input is glued into a query by hand.

8 · A complete screen, end to end (Login)

Two files make one screen. First, **what it looks like**:

SRC/ITEMS/LOGIN.ASPX (THE APPEARANCE)

```
1 <%@ Page Language="VB" AutoEventWireup="false"
2     CodeBehind="Login.aspx.vb" Inherits="ITEMS.LoginPage" %>
3 <asp:TextBox ID="txtUser" runat="server" placeholder="e.g. admin" />
4 <asp:TextBox ID="txtPass" runat="server" TextMode="Password" />
5 <asp:Button ID="btnLogin" runat="server" Text="Sign in →" OnClick="btnLogin_Click" />
```

LINE	WHAT IT MEANS IN PLAIN WORDS
1–2	"This page is written in Visual Basic; its behaviour lives in `Login.aspx.vb`."
3	A box to type the user id into. `runat="server"` is the key phrase: it means the *server* can read this box, not just the browser.
4	The same, but `TextMode="Password"` makes the browser show dots instead of letters.
5	A button. When clicked, run the procedure named `btnLogin_Click` on the server.

Second, **what it does**:

SRC/ITEMS/LOGIN.ASPX.VB (THE BEHAVIOUR)

```
1 Protected Sub btnLogin_Click(sender As Object, e As EventArgs)
2     Dim fullName = Security.Authenticate(txtUser.Text, txtPass.Text)
3     If String.IsNullOrEmpty(fullName) Then
4         litErr.Text = "Invalid user id or password. Please try again."
5         Return
6     End If
7     FormsAuthentication.SetAuthCookie(fullName, False)
8     Dim target = Request.QueryString("ReturnUrl")
9     Response.Redirect(If(String.IsNullOrEmpty(target), "~/Default.aspx", target))
10End Sub
```

LINE	WHAT IT MEANS IN PLAIN WORDS
1	"When the Sign in button is clicked, do this."
2	Hand the typed user id and password to the security checker from §6.
3–5	If nothing came back, the details were wrong: show one message and stop. Note it does **not** say "wrong password" — telling an attacker which half was right is a security weakness.
7	Success. Issue a **cookie** — a signed pass the browser shows on each later page, so the user does not log in again for every click.
8–9	If the user was bounced here from a page they wanted, send them back to it; otherwise, to the dashboard.

> **HOW EVERY OTHER PAGE STAYS PROTECTED**
>
> One rule in `Web.config` — `<deny users="?" />` — means "anyone not signed in is refused", applied to the whole site at once. The `?` stands for an anonymous visitor.

9 · The Client module — adding a client

What it does: checks the typed details, refuses duplicates, then writes one new row.

SRC/ITEMS/CLIENTEDIT.ASPX.VB

```
1 Protected Sub btnSave_Click(sender As Object, e As EventArgs)
2     If Not Page.IsValid Then Return
3     Dim pan = txtPan.Text.Trim().ToUpperInvariant()
4     If Not IsEdit AndAlso ClientRepository.Exists(pan) Then
5         litErr.Text = "A client with this PAN already exists."
6         Return
7     End If
8     Dim c As New Client With {.Pan = pan, .Name = txtName.Text.Trim(), ...}
9     If IsEdit Then ClientRepository.Update(c) Else ClientRepository.Insert(c)
10    Response.Redirect("Clients.aspx")
11End Sub
```

LINE	WHAT IT MEANS IN PLAIN WORDS
2	Were the on-screen checks satisfied (PAN pattern, name present)? If not, stop — the page already shows the messages.
3	Tidy the PAN: remove stray spaces and force capitals, so `abcde1234f` and `ABCDE1234F` are stored identically.
4–7	If this is a new client, make sure the PAN is not already taken, and say so clearly if it is.
8	Gather everything typed on screen into one tidy **Client** object — a parcel of related facts passed around as a unit.
9	The same button serves both jobs: update an existing row, or insert a new one.
10	Go back to the list, where the change is now visible.

THREE LAYERS OF DEFENCE

The same PAN rule is enforced in the browser (instant feedback), in the VB code (line 4), and in the database (§4.1). Each layer alone could be bypassed; together they cannot.

10 · The Return module — and the one business rule

The specification states a revised return may only be filed if an original return already exists for that client and year. Here is that sentence, turned into code.

SRC/ITEMS/RETURNEDIT.ASPX.VB

```
1 If Not IsEdit AndAlso rec.ReturnType = 1 AndAlso
2    Not ReturnRepository.HasOriginal(rec.Pan, rec.AyFrom, rec.AyTo) Then
3     litErr.Text = "An original return must be filed before a revised one."
4     RenderComputation(rec)
5     Return
6 End If
7 ReturnRepository.Save(rec, IsFirm(), Not IsEdit)
```

LINE	WHAT IT MEANS IN PLAIN WORDS
1	Three conditions, all of which must be true to raise the objection: this is a *new* filing, it is marked *revised* (type 1), and…
2	…no original return can be found for that client and year.
3–5	Explain the problem in the user's language, redraw the tax panel so nothing they typed is lost, and stop before saving.
7	Rule satisfied — save. `IsFirm()` tells the tax engine which rate to use; `Not IsEdit` tells the repository whether to insert or update.

WHERE THE TAX IS WORKED OUT

Notice the screen never calculates anything itself. It gathers numbers and calls the tax engine (§5). Because the engine is one shared piece of code, the web edition and the Visual Basic edition can never drift apart.

11 · Firm accounts — one design for three screens

Trading Account, Profit & Loss Account and Balance Sheet look different but behave identically: two columns of money that must add up to the same total. Rather than write three near-identical screens, the project **describes** each one as data and lets one screen serve all three.

SRC/ITEMS/APP_CODE/FIRMACCOUNTREPOSITORY.VB

```
1 Return New FirmAccountSpec With {
2     .Table = "BALANCE_SHEET", .Title = "Balance Sheet",
3     .LeftHead = "Liabilities & Capital", .RightHead = "Assets",
4     .LeftCols = {"BILLS_PAYABLE", "SUNDRY_CREDITORS", "LOANS", ...},
5     .RightCols = {"CASH_IN_HAND", "CASH_AT_BANK", "INVESTMENTS", ...}}
```

LINE	WHAT IT MEANS IN PLAIN WORDS
1	A **specification**: a description of one of the three account books, written as plain data rather than as code.
2	Which table to store it in, and what to call it on screen.
3	The heading over each column. For a balance sheet these are "Liabilities" and "Assets"; for the other two they are "Dr." and "Cr.".
4–5	Which money fields belong on the left, and which on the right. The screen builds its input boxes by reading these lists.

WHY THIS IS GOOD DESIGN

Adding a new field to the balance sheet means adding one name to a list — the input box, the total, and the balance check all follow automatically. One screen (`FirmAccount.aspx`) serves three modules, so a fix applies to all three at once.

12 · The web edition

The live demo is the same system rewritten so it needs no server and no Oracle — that is what lets anyone open a link and use it. The differences are deliberate and small:

CONCERN	PRODUCTION (VB + ORACLE)	WEB EDITION
Where data lives	Oracle database on a server	`localStorage` — a private store inside your own browser
Tax rules	`TaxEngine.vb`	`tax.js` — identical logic
Who can see the data	Everyone who logs in	Only you, on your device

> **CONSEQUENCE WORTH KNOWING**
> Because the web edition stores data in your own browser, two people opening the link each get their own private copy. That is ideal for a demo — nobody can spoil anyone else's walkthrough — but it is not shared storage. The Oracle edition is what provides that.

The same staircase calculation, in the web edition's language:

WEBAPP/JS/TAX.JS

```
function slabTax(income) {
  let tax = 0, lower = 0;
  for (const s of SLABS) {
    if (income > lower) {
      const band = Math.min(income, s.upto) - lower;
      tax += band * s.rate;
      lower = s.upto;
    } else break;
  }
  return Math.round(tax);
}
```

Compare this with §5.1: the words differ (`function` instead of `Function` , `break` instead of `Exit For`) but the reasoning is line-for-line the same — which is exactly why both editions always produce ₹1,79,400 for a ₹12,00,000 salary.

13 · Code review — what was checked and what was fixed

Before release the whole project was reviewed line by line. The web edition was additionally **driven in a real browser** — signing in, adding data, computing tax, generating reports, and checked at both desktop and mobile sizes. The issues below were found and corrected.

13.1 Defects found and fixed

#	ISSUE	EFFECT IF SHIPPED	FIX
1	Missing `.designer.vb` files	The project would not compile at all — every on-screen control was undeclared.	Generated a designer file per page declaring all controls; registered them in the project.
2	`Imports System.Web.UI.WebControls` missing in 4 code-behinds	Compile errors on `TextBox`, `GridViewCommandEventArgs`.	Added the required imports to each file.
3	`Data.DataTable` in Reports	Compile error: `Data` resolved to the project's own `ITEMS.Data`, not `System.Data`.	Qualified it fully as `System.Data.DataTable`.
4	`AutoEventWireup="true"` together with `Handles Me.Load`	**Every page would load twice** — double database reads on every request.	Set `AutoEventWireup="false"` (the correct convention for VB).
5	Oracle driver version mismatch in `Web.config`	The app could fail at start-up unable to load the Oracle library.	Aligned the declared version with the installed package.
6	Unbalanced demo balance sheets	The demo opened showing a red "difference" warning — bad first impression.	Corrected the sample figures so assets equal liabilities.
7	Credentials invisible on the closing slide	White text on a white card — the demo video showed blank boxes.	Set an explicit dark colour on the card.
8	Temporary calculation object saved with each return	Harmless today, but stored junk that would confuse future maintenance.	Removed the stray assignment.

13.2 Verified working

AREA	HOW IT WAS CHECKED	RESULT
Tax computation	₹12,00,000 salary, computed by hand vs. the app	Both give ₹1,79,400 ✓
Sign in / sign out	Driven in a real browser, both accounts	Works ✓
Client add / edit / delete	Exercised with validation and duplicates	Works ✓
Original & revised returns	Rule tested with and without an original	Correctly enforced ✓
Firm accounts	Totals and balance status re-checked	Both firms balance ✓
Reports	Return history generated for a client	Shows original + revised ✓
Mobile & dark mode	Rendered at 390 px and in dark theme	Correct ✓

13.3 Honest limitations

- The Visual Basic edition was reviewed by inspection and corrected, but it has **not been compiled or run** here — that needs Windows, .NET and an Oracle instance. Build it once in Visual Studio before the client demo.
- Tax rates are illustrative (old regime, AY 2024-25). Verify against the current Finance Act before real use.
- The demo passwords are public by design. Change them before any real deployment.

14 · Glossary

TERM	PLAIN MEANING
Table	A grid of data. Columns are fields; rows are records.
Primary key	The field that uniquely identifies a row — here, the PAN.
Foreign key	A link from one table to another, e.g. a return points at its client.
Cascade delete	Deleting a client automatically removes everything attached to them.
Constraint	A rule the database enforces itself, such as the PAN pattern.
Trigger	Code the database runs by itself when data changes — used here for the audit log.
SQL	The language used to ask a database questions.
SQL injection	An attack where typed text is mistaken for commands. Prevented by parameters.
Parameter	A value sent to the database separately from the instruction — the safe way.
Hash (SHA-256)	A one-way fingerprint of text. Used so passwords are never stored.
Code-behind	The file holding a screen's behaviour, separate from its appearance.
Postback	The page sending itself back to the server when a button is pressed.
Repository	The one place that knows how to read/write a particular table.
Assessment year	The year in which the previous year's income is assessed to tax.
TDS	Tax Deducted at Source — tax already withheld, subtracted at the end.
Cess	An extra 4% charged on the tax, funding health and education.
Surcharge	An additional percentage on the tax itself for high incomes.
Rebate 87A	A reduction in tax for smaller total incomes.